

# Praktikumsbericht zu Versuch 10

Von Daniel Baer (3235621)

## 1. Cantorsches Diagonalargument

### a) Diagonalfolge konstruieren

$$f_1(1) = 0$$

$$f_2(2) = 0$$

$$f_3(3) = 1$$

$$f_4(4) = 0$$

$$f_5(5) = 0$$

Daraus ergibt sich die Folge  $d$ : 1 1 0 1 1

### b) Widerspruch

Wäre  $d$  die  $k$ -te Folge der Liste ( $d = f_k$ ), gilt:  $d(k) = f_k(k)$

Außerdem würde durch die Definition von  $d$  gelten:  $d(k) = 1 - f_k(k)$

Daraus würde folgen:  $f_k(k) = 1 - f_k(k)$

So entsteht der Widerspruch und  $d$  ist nicht die  $k$ -te Folge der Liste.

### c) Schlussfolgerung

Jedes Python Programm besteht aus endlich vielen Befehlen und mehrere Programme können lexikographisch geordnet werden, sie sind abzählbar.

In unendlich binären Folgen kann sich immer ein Zeichen zu einer anderen Folge unterscheiden, die folgen sind nicht abzählbar oder durch ein Programm berechenbar.

## 2. Das Halteproblem

### a) Widerspruchsbeweis

Falls  $H(Z, y) = 0$  (1. Fall), terminiert das Programm nach Definition.

Das passiert nur wenn  $H(Z, y) = 1$  (2. Fall), dann läuft das Programm allerdings nach Definition weiter.

So entsteht ein Widerspruch.

### b) Konsequenz für die Praxis

Es gibt keine allgemeine Möglichkeit zu bestimmen, ob und wann ein Programm terminiert.

Ein Antivirus-Programm kann beispielsweise nicht nachvollziehen, ob ein Programm endlos weiterläuft.

Ebenfalls hat ein Compiler Probleme das Programm zu optimieren, wenn nicht erkannt wird, ob eine Funktion/Schleife beendet wird.

Diese Programme müssen mit Richtlinien und Beschränkungen arbeiten, um solche Fälle abzufangen.

### 3. Komplexitätsklassen

#### a) Einordnung

1. P, mit linearem Scan aller Zahlen.
2. nicht berechenbar, es kann nicht sicher bestimmt werden, ob ein Programm terminiert (Halteproblem).
3. NP-vollständig, es müssen im schlimmsten Fall alle Belegungen geprüft werden.
4. P, die Formel kann mit der Belegung ausgeführt werden.
5. NP-vollständig, es müssen im schlimmsten Fall alle Pfade berechnet werden.
6. P, die Länge kann polynomial berechnet werden.

#### b) Ackermann-Funktion

##### 1. Berechnung per Hand

$$\begin{aligned} & A(1,2) \\ &= A(0, A(1,1)) \\ &= A(0, A(0, A(1,0))) \\ &= A(0, A(0, A(0,1))) \\ &= A(0, A(0,2)) \\ &= A(0,3) \\ &= 4 \end{aligned}$$

$$\begin{aligned} & A(2,2) \\ &= A(1, A(2,1)) \\ &= A(1, A(1, A(2,0))) \\ &= A(1, A(1, A(1,1))) && \text{(von oben } A(1,1) = 3) \\ &= A(1, A(1,3)) \\ &= A(1, A(0, A(1,2))) && \text{(von oben } A(1,2) = 4) \\ &= A(1, A(0,4)) \\ &= A(1,5) \\ &= A(0, A(1,4)) \\ &= A(0, A(0, A(1,3))) && \text{(bereits berechnet } A(1,3) = 5) \\ &= A(0, A(0,5)) \\ &= A(0,6) \\ &= 7 \end{aligned}$$

## 2. Implementierung

```
def ack(m: int, n: int) -> int:
    if m == 0:
        return n + 1
    if n == 0:
        return ack(m - 1, 1)
    return ack(m - 1, ack(m, n - 1))
```

## 3. Berechenbarkeit der Ackermann-Funktion

Die Ackermann-Funktion ist zwar berechenbar, aber nicht in polynomialer Zeit.

Die Berechnungsdauer Ackermann-Funktion wächst mit steigenden Argumenten extrem schnell.

So ist die Funktion nicht in P enthalten.